

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г. В. Плеханова»
МОСКОВСКИЙ ПРИБОРОСТРОИТЕЛЬНЫЙ ТЕХНИКУМ

ОТЧЕТ

по производственной практике

ПП.04.01 Производственная практика

по специальности 09.02.09 «Веб-разработка»

Тема: разработка веб-мессенджера «БренксЧат»

Студент: _____

Группа: ВТ-Х-ХХ

Руководитель практической подготовки от профильной организации:

Руководитель практической подготовки от техникума:

Москва, 2026

Содержание

Результаты анализа проекта

Введение

1 Цели и задачи практики

2 База практики

3 Содержание практики

4 Охрана труда

Вывод

Список использованных источников

Результаты анализа проекта

В ходе анализа проекта установлено, что разработанное приложение представляет собой веб-мессенджер «БренксЧат» с клиент-серверной архитектурой. Клиентская часть реализована как одностраничное приложение на React и TypeScript, серверная часть — на Node.js, Express и Socket.IO. Для постоянного хранения данных используется MySQL-совместимая база данных через библиотеку mysql2; при отсутствии MySQL предусмотрен файловый вариант хранения состояния в JSON-файле.

- Назначение проекта: обеспечение обмена сообщениями в реальном времени, работы личных чатов, групп, каналов, медиа-сообщений, голосовых и видеозвонков, а также администрирования пользователей.
- Клиентские технологии: React 18, TypeScript, Vite, Tailwind CSS, React Router, Socket.IO Client, Web APIs браузера, Service Worker и PWA manifest.
- Серверные технологии: Node.js, Express, Socket.IO, MySQL2, JSON Web Token, Argon2id, Helmet, CORS, express-rate-limit, Nodemailer, Web Push, UUID.
- Криптографические и защитные механизмы: хэширование паролей Argon2id, cookie-сессии с JWT, ограничение частоты авторизационных запросов, подтверждение почты кодом, резервное копирование E2EE-ключей, шифрование текстов личных сообщений через libsodium.
- Архитектура: SPA-клиент, REST API, WebSocket-слой для realtime-событий, слой хранения состояния, MySQL-схема, Nginx reverse proxy с HTTPS и отдельные deploy-конфигурации.
- Основные клиентские модули: AuthContext, MessengerContext, CallContext, ThemeContext, ChatWallpaperContext; компоненты ChatList, ChatWindow, MessageBubble, MessageInput, NewChatModal, ProfileModal, PhotoViewer, VoiceVideoMedia.

- Основные серверные модули: `index.ts`, `socketHandlers.ts`, `store.ts`, `persist.ts`, `auth.ts`, `sessions.ts`, `password.ts`, `emailAuth.ts`, `email.ts`, `e2eeDevices.ts`, `pushNotifications.ts`.
- Структура базы данных: `users`, `chats`, `chat_participants`, `messages`, `user_chat_muted`, `user_chat_pinned`, `push_subscriptions`, `auth_sessions`, `user_e2ee_devices`, `user_e2ee_key_backups`, `app_state`.
- Тестирование: в серверной части обнаружены автоматизированные тесты для Argon2id-паролей, сессий, E2EE-устройств и миграции учетных данных; команда `npm test -w server` выполнена успешно, 8 тестов пройдено.
- Сборка: команда `npm run build` выполнена успешно; клиент собран Vite, сервер скомпилирован TypeScript-компилятором.

Таблица 1 — Данные, отсутствующие в проектных материалах

Данные	Что найдено	Комментарий для заполнения
ФИО студента, номер курса, точная группа	В шаблонах оставлены поля-заполнители.	Вписать фактические данные студента и группы перед сдачей отчета.
Название профильной организации	В проекте и шаблонах не указано.	Использовать формулировку о прохождении практики в рамках учебного проекта либо вписать организацию, если она была назначена техникумом.
Точные характеристики компьютера	В репозитории отсутствует паспорт оборудования.	Заполнить по фактическому ПК: процессор, ОЗУ, накопитель, монитор, периферия.
Точная версия MySQL на сервере	В коде используется <code>mysql2</code> и MySQL-совместимый синтаксис, но версия СУБД не зафиксирована.	Указать версию из <code>phpMyAdmin</code> или команды <code>SELECT VERSION()</code> .
ФИО руководителей практики	В предоставленных документах поля пустые.	Заполнить по данным образовательной организации и базы практики.

Введение

Производственная практика ПП.04.01 по специальности 09.02.09 «Веб-разработка» была направлена на закрепление профессиональных навыков разработки веб-приложений и получение практического опыта полного цикла создания программного продукта. В качестве практического задания был разработан мессенджер «БренксЧат», предназначенный для обмена сообщениями, управления чатами, отправки медиафайлов, организации голосовой и видеосвязи, а также администрирования пользователей.

Сведения о конкретной профильной организации в исходных документах и проектных файлах отсутствуют. В связи с этим в отчете используется формулировка о прохождении практики в рамках учебного проекта на базе собственного рабочего места студента и серверной среды, подготовленной для размещения веб-приложения. Подразделение практики условно определяется как учебная проектная среда веб-разработки, в которой выполнялись анализ требований, проектирование, программирование, тестирование, отладка и подготовка проекта к развертыванию.

Направление деятельности в рамках практики соответствовало профилю веб-разработчика: создание клиентской части приложения, разработка серверного REST API, настройка обмена данными в реальном времени через WebSocket, проектирование базы данных, настройка авторизации, работа с безопасностью и подготовка деплойных конфигураций для VPS-сервера.

В процессе разработки использовались технические и программные средства, фактически обнаруженные в проекте: Node.js, npm workspaces, TypeScript, React, Vite, Tailwind CSS, Express, Socket.IO, MySQL, Nginx, systemd, Git, Service Worker, а также средства автоматизированной проверки: TypeScript compiler, Vite production build и тесты Node.js через tsx --test.

[Вставить скриншот: главный экран мессенджера «БренксЧат» со списком чатов и открытой перепиской]

1 Цели и задачи практики

Целью производственной практики являлось комплексное освоение практических навыков веб-разработки путем самостоятельного создания полнофункционального клиент-серверного приложения. Разработка мессенджера позволила применить знания по программированию, проектированию интерфейсов, организации сетевого взаимодействия, работе с базами данных, обеспечению безопасности и сопровождению приложения.

В ходе практики были поставлены следующие задачи:

- изучить предметную область мессенджеров и определить состав пользовательских сценариев;
- спроектировать клиент-серверную архитектуру приложения;
- разработать структуру данных для пользователей, чатов, сообщений, сессий, push-подписок и E2EE-устройств;
- реализовать регистрацию, вход, подтверждение почты, сброс пароля и режим «запомнить меня»;
- реализовать обмен сообщениями в реальном времени через Socket.IO;
- реализовать личные чаты, группы, каналы, избранное, поиск, ответы, реакции, редактирование, удаление и пересылку сообщений;
- добавить отправку изображений, файлов, голосовых сообщений и видеокружков;
- реализовать голосовые и видеозвонки на базе WebRTC-сигналинга;
- настроить хранение данных в MySQL и подготовить конфигурации для серверного развертывания;
- провести тестирование, исправить ошибки и подготовить проектную документацию.

В результате выполнения работы были сформированы профессиональные компетенции, связанные с разработкой программного кода, проектированием интерфейсов, взаимодействием с серверной частью, использованием баз данных, анализом ошибок, тестированием и сопровождением веб-приложения.

Выполненная работа напрямую связана с профессиональной деятельностью веб-разработчика, так как охватывает полный цикл создания современного веб-сервиса: от структуры проекта и дизайна интерфейса до серверного API, БД, безопасности и деплоя.

2 База практики

База практики представлена учебной проектной средой, включающей локальное рабочее место разработчика, репозиторий исходного кода и серверную инфраструктуру для размещения веб-приложения. Точные характеристики аппаратного обеспечения в файлах проекта не зафиксированы. По структуре путей рабочей директории (/Users/rozles/...) можно сделать вывод, что разработка велась в macOS-совместимой локальной среде. Для развертывания подготовлены конфигурации Ubuntu/VPS: Nginx, systemd, MySQL и резервное копирование.

Аппаратное обеспечение, которое необходимо указать при финальном заполнении отчета: персональный компьютер или ноутбук разработчика, монитор, клавиатура, мышь, сетевой адаптер и доступ к интернету. Для серверной части использовался VPS с Linux/Ubuntu, что подтверждается конфигурациями `deploy/systemd/brenkschat.service` и `deploy/nginx/silentx.ru.conf`. Точные параметры CPU, RAM и диска VPS в проектных файлах не указаны.

Таблица 2 — Используемые технические и программные средства

Тип средства	Название	Назначение
Язык программирования	TypeScript	Основной язык клиентской и серверной разработки; используется в <code>client/src</code> и <code>server/src</code> .
Среда выполнения	Node.js	Запуск серверной части, сборочных скриптов, тестов и production-приложения.
Менеджер пакетов	npm workspaces	Управление зависимостями <code>client</code> и <code>server</code> из корневого <code>package.json</code> .
Клиентский фреймворк	React 18	Построение интерфейса мессенджера, страниц входа,

		чатов и модальных окон.
Маршрутизация	React Router	Разделение маршрутов /login, / и /admin в client/src/App.tsx.
Сборщик	Vite	Локальный dev-сервер на порту 5173, проху /api и /socket.io, production-сборка клиента.
CSS-фреймворк	Tailwind CSS	Стилизация интерфейса, темы, адаптивность, анимации сообщений и модальных окон.
Серверный фреймворк	Express	REST API для авторизации, чатов, пользователей, админ-панели, push и E2EE.
Realtime	Socket.IO	Обмен сообщениями, typing, presence, реакции, звонки и события чатов в реальном времени.
База данных	MySQL	Постоянное хранение пользователей, чатов, сообщений, сессий и служебных данных.
Драйвер БД	mysql2	Подключение Node.js-сервера к MySQL через DATABASE_URL.
Авторизация	jsonwebtoken	Формирование JWT с идентификатором пользователя и сессии.
Хэширование паролей	argon2	Безопасное хранение паролей с алгоритмом Argon2id.
Почта	Nodemailer / Resend API	Отправка кодов подтверждения входа, регистрации, привязки почты и сброса пароля.
Push-уведомления	web-push, Service Worker	Уведомления о новых сообщениях и входящих звонках.
Шифрование	libsodium-wrappers, Web Crypto API	E2EE-текстовые сообщения, Curve25519-ключи, резервная копия ключей AES-GCM/PBKDF2.
Звонки	WebRTC	Голосовые и видеозвонки; ICE/STUN/TURN задаются через API /api/calls/ice-servers.

Веб-сервер	Nginx	HTTPS reverse proxy для API, Socket.IO, статических файлов и phpMyAdmin.
Сервисная среда	systemd	Автозапуск и hardening Node.js-сервера на VPS.
Контроль версий	Git	История коммитов проекта; в журнале есть начальные коммиты и конфигурация Railway.
Тестирование	tsx --test, TypeScript compiler, Vite build	Автоматизированные серверные тесты, проверка типов, production-сборка.
Документация	README.md, DESKTOP_RELEASE.md, deploy/*.conf	Описание запуска, структуры, desktop-направления и серверного развертывания.
Дополнительный desktop-клиент	Flutter / Dart	В каталоге desktop находится нативный MVP-клиент для macOS/Windows, не являющийся webview-оберткой.
Пакетирование	Electron / electron-builder	В корневом package.json сохранены скрипты сборки desktop-упаковки для macOS/Windows/Linux.

Структура вычислительной среды включает локальный клиент, сервер Node.js, СУБД MySQL и внешний reverse proxy. В режиме разработки клиент запускается Vite на порту 5173, а API и Socket.IO проксируются на сервер. В production Nginx обслуживает статические файлы и перенаправляет API/WebSocket-запросы на Node.js-сервис. Для проектирования использовались TypeScript-интерфейсы в server/src/types.ts и client/src/types/index.ts, структура каталогов компонентов и контекстов, а также SQL-схема, создаваемая в persist.ts, sessions.ts и e2eeDevices.ts.

3 Содержание практики

Содержание практики включало последовательную разработку мессенджера, начиная с анализа требований и заканчивая тестированием, документированием и подготовкой инфраструктуры для развертывания. Ниже

приведено описание основных этапов с указанием использованных технологий, подтверждающих файлов и результата.

Таблица 3 — Этапы выполнения производственной практики

Этап	Что было сделано	Файлы проекта	Результат
Анализ требований	Определены основные сценарии: регистрация, вход, подтверждение почты, личные чаты, группы, каналы, медиа, звонки, администрирование.	README.md, client/src/pages/LoginPage.tsx, client/src/pages/MessengerPage.tsx, client/src/pages/AdminPage.tsx	Сформирован состав функций мессенджера и пользовательских ролей.
Проектирование системы	Выбрана архитектура SPA + REST API + WebSocket. Клиент разделен на страницы, компоненты и контексты; сервер — на API, realtime-обработчики, store и persistence.	client/src/App.tsx, client/src/context/* .tsx, server/src/index.ts, server/src/socketHandlers.ts, server/src/store.ts	Получена расширяемая структура проекта с отдельными зонами ответственности.
Проектирование БД	Спроектированы таблицы users, chats, messages, chat_participants, auth_sessions, push_subscriptions, E2EE-таблицы и служебные таблицы.	server/src/persist.ts, server/src/sessions.ts, server/src/e2eeDevices.ts, server/src/types.ts	Создана MySQL-схема для постоянного хранения данных мессенджера.
Разработка серверной части	Реализованы REST-эндпоинты Express, Socket.IO-события, проверка прав, рассылка уведомлений, нормализация медиа и обработка звонков.	server/src/index.ts, server/src/socketHandlers.ts, server/src/pushNotifications.ts, server/src/email.ts	Сервер обеспечивает API, realtime-обмен, авторизацию и интеграцию с инфраструктурой.
Разработка клиентской части	Созданы страницы входа, мессенджера и админ-панели; реализованы компоненты чата, строки ввода, профиля, просмотра фото, тем и модальных окон.	client/src/pages/* .tsx, client/src/components/* .tsx, client/src/index.css, client/tailwind.config.js	Получен адаптивный пользовательский интерфейс с темной и светлой темой.
Авторизация и	Добавлены регистрация,	server/src/auth.ts,	Пользователь получает

аутентификация	вход, подтверждение кода из письма, сброс пароля, cookie-сессии, JWT, режим remember me и роль администратора.	server/src/password.ts, server/src/emailAuth.ts, server/src/sessions.ts, client/src/contexts/AuthContext.tsx	защищенную сессию, а администратор — доступ к отдельной панели.
Обмен сообщениями	Реализованы отправка, редактирование, удаление, реакции, ответы, пересылка, закрепление, прочтение, поиск, typing и presence.	server/src/socketHandlers.ts, client/src/contexts/MessengerContext.tsx, client/src/components/MessageBubble.tsx, client/src/components/MessageInput.tsx	Сообщения доставляются в реальном времени и синхронизируются между участниками чата.
Работа с медиа и звонками	Добавлены изображения, файлы, голосовые сообщения, видеокружки, WebRTC-звонки, уведомления о звонках и настройка ICE-серверов.	client/src/components/VoiceVideoMedia.tsx, client/src/contexts/CallContext.tsx, client/src/components/PhotoViewer.tsx, server/src/index.ts	Мессенджер поддерживает мультимедийные сценарии общения.
Работа с базой данных	Реализованы загрузка и сохранение состояния в MySQL, миграция из app_state/JSON, периодическое сохранение, резервное копирование.	server/src/persist.ts, deploy/backup/brenkschat-backup.mjs, server/scripts/*.mjs	Данные приложения могут храниться на сервере и переноситься между средами.
Тестирование и отладка	Проверены password/session/E2EE/store-сценарии, выполнены npm test -w server и npm run build.	server/src/*.test.ts, package.json, server/package.json, client/package.json	8 серверных тестов пройдено, production-сборка клиента и сервера выполнена успешно.
Документирование и деплой	Подготовлены README, desktop-документация, systemd service, Nginx-конфигурация, резервное копирование и env-пример.	README.md, DESKTOP_RELEASE.md, server/.env.example, deploy/nginx/silentx.ru.conf, deploy/systemd/brenkschat.service	Проект имеет инструкции запуска и основу для размещения на VPS.

3.1 Анализ требований

На первом этапе была определена предметная область проекта. Мессенджер должен был обеспечивать быстрое общение пользователей, поддерживать персональные и групповые сценарии, иметь современный интерфейс, работать в

браузере и поддерживать дальнейшее серверное развертывание. README.md фиксирует исходную идею Telegram-style мессенджера на React + TypeScript и Node.js + Express + Socket.IO. В процессе развития проекта функциональность была расширена: появились подтверждение почты, MySQL, E2EE-слой, звонки, push-уведомления, PWA-манифест и административная панель.

[Вставить скриншот: страница входа с вкладками входа, регистрации, сброса пароля и кодом подтверждения]

3.2 Проектирование системы

Архитектура приложения построена по клиент-серверной модели. Клиентская часть находится в каталоге client и реализует интерфейс пользователя. Серверная часть находится в каталоге server и отвечает за API, обработку realtime-событий и сохранение данных. В client/src/App.tsx описаны маршруты /login, / и /admin. Страница MessengerPage объединяет CallProvider, ChatWallpaperProvider и MessengerProvider, а также основные компоненты интерфейса: список чатов, окно переписки, шапку чата, строку ввода и модальные окна.

Серверная часть разделена на index.ts, где объявлены HTTP-эндпоинты Express, socketHandlers.ts, где обрабатываются события Socket.IO, store.ts, где описаны операции над состоянием, и persist.ts, где реализовано постоянное хранение данных. Такой подход позволяет отделить транспортный слой от бизнес-логики и от слоя работы с базой данных.

3.3 Проектирование базы данных

В проекте предусмотрено два варианта хранения: резервный файловый JSON и основной MySQL-режим, который включается при PERSIST_BACKEND=mysql и наличии DATABASE_URL. MySQL-схема создается программно при запуске через ensureMysqlSchema, initializeSessionStore и initializeE2eeDevices. Это снижает риск ручной ошибки при развертывании приложения.

Таблица 4 — Структура базы данных проекта

Таблица	Назначение	Основные поля
users	Пользователи	id, username, password, email, email_verified, avatar_url, display_name, bio, phone, birth_date, is_admin, banned, privacy
chats	Чаты	id, type, name, avatar_url, last_message, unread, last_read_at, pinned_message_id, channel_owner_id
chat_participants	Связь пользователей и чатов	chat_id, user_id, position; внешний ключ на chats(id) с ON DELETE CASCADE
messages	Сообщения	id, chat_id, sender_id, text, encrypted_text, image_url, media_*, created_at, deleted, edited_at, reply_to_message_id, reactions
user_chat_muted	Отключенные уведомления	user_id, chat_id
user_chat_pinned	Закрепление чатов в списке	user_id, chat_id
push_subscriptions	Push-подписки браузеров	user_id, endpoint, data
auth_sessions	Активные сессии	id, user_id, created_at, expires_at
user_e2ee_devices	Публичные ключи E2EE-устройств	user_id, device_id, public_key, created_at, last_seen_at
user_e2ee_key_backups	Резервные копии ключей E2EE	user_id, version, salt, iv, ciphertext, iterations, updated_at
app_state	Совместимость со старым форматом	id, data, updated_at

[Вставить скриншот: фрагмент phpMyAdmin или схемы MySQL с таблицами users, chats и messages]

3.4 Разработка серверной части

Серверная часть реализована на Express и Socket.IO. В server/src/index.ts объявлены маршруты для регистрации, входа, подтверждения входа кодом, выхода, сброса пароля, управления профилем, поиска пользователей, создания чатов, групп и каналов, получения сообщений, работы с E2EE-устройствами, push-подписками и админ-панелью. В server/src/socketHandlers.ts реализованы

события `send_message`, `edit_message`, `delete_message`, `toggle_reaction`, `forward_messages`, `mark_read`, `typing` и `call_signal`.

Для защиты API применяются `Helmet`, `CORS`, `rate limiting` и проверка авторизации. Конфигурация `server/.env.example` предусматривает `CLIENT_ORIGINS`, ключи `VAPID`, провайдера почты и параметры `SMTP/Resend`. В production-среде Nginx проксирует `/api` и `/socket.io` на Node.js-сервер, а также обслуживает статические файлы клиентской сборки.

3.5 Разработка клиентской части

Клиентская часть реализована как одностраничное React-приложение. Компонентная структура позволяет разделить ответственность между отдельными элементами интерфейса. `ChatList` отвечает за список чатов и поиск, `ChatWindow` отображает переписку, `MessageBubble` отвечает за внешний вид сообщения и действия с ним, `MessageInput` реализует ввод текста, вложения, вставку изображений из буфера, запись голосовых сообщений и видеокружков. `PhotoViewer` используется для просмотра изображений, `VoiceVideoMedia` — для голосовых и видео-сообщений.

Для управления состоянием используются `React Context`: `AuthContext` хранит текущего пользователя и методы авторизации; `MessengerContext` синхронизирует чаты, сообщения, `typing`, реакции и `Socket.IO`; `CallContext` управляет голосовыми и видеозвонками; `ThemeContext` отвечает за светлую и темную тему; `ChatWallpaperContext` хранит настройки фонов чата. Стилизация реализована `Tailwind CSS` и дополнительными CSS-классами в `client/src/index.css`.

[Вставить скриншот: модальное окно создания нового чата, группы или канала в стеклянном стиле]

3.6 Авторизация и аутентификация

Регистрация и вход реализованы с учетом подтверждения электронной почты. На сервере `emailAuth.ts` создает одноразовые коды, а `email.ts` отправляет письма через `Resend`, `SMTP` или выводит письмо в консоль в режиме разработки.

Пароли хранятся не в открытом виде: password.ts использует Argon2id, проверяет минимальную длину и поддерживает миграцию устаревших учетных данных.

После успешного входа сервер выдает JWT, связанный с записью в auth_sessions. Сессии могут быть короткими или длительными, если пользователь выбрал режим «запомнить меня». В auth.ts проверяется cookie brends_session или Bearer-токен, наличие активной сессии, существование пользователя и отсутствие блокировки. Для администратора предусмотрена дополнительная проверка requireAdmin.

3.7 Реализация обмена сообщениями

Обмен сообщениями построен на Socket.IO. При подключении пользователь присоединяется к комнатам своих чатов и комнате user:<id>. При отправке сообщения сервер проверяет участие пользователя в чате, права отправки в канал, корректность зашифрованного текста и параметры медиа. После сохранения сообщение рассылается участникам, обновляется preview последнего сообщения, счетчики непрочитанных и push-уведомления.

Для удобства общения реализованы ответы на сообщения, реакции, пересылка, редактирование, удаление, закрепление, мьют чата, закрепление чата в списке, отметка прочтения и индикатор печати. В личных чатах предусмотрен слой E2EE для текста: клиент формирует envelope с алгоритмом crypto_box_curve25519xsalsa20poly1305, а сервер хранит его как JSON без расшифровки текста.

[Вставить скриншот: пример переписки с текстовым сообщением, реакциями, ответом и медиавложением]

3.8 Работа с базой данных

Слой persist.ts преобразует внутреннее состояние приложения в нормализованные таблицы MySQL и обратно. При чтении состояния сервер загружает пользователей, чаты, участников, сообщения, настройки mute/pin и push-подписки. При сохранении состояние записывается в соответствующие

таблицы. Для совместимости сохранена таблица `app_state`, которая содержит JSON-снимок старого формата.

Для эксплуатации подготовлен backup-скрипт `deploy/backup/brenkschat-backup.mjs`. Он выполняет `mysqldump` с параметрами безопасного дампа, архивирует исходный код без `node_modules`, `client/dist`, `server/dist`, `server/data` и `.git`, сохраняет важные конфигурационные файлы, формирует `SHA256SUMS` и `metadata.json`, а также удаляет старые резервные копии старше установленного срока хранения.

3.9 Тестирование и исправление ошибок

В проекте обнаружены автоматизированные тесты `server/src/e2eeDevices.test.ts`, `server/src/password.test.ts`, `server/src/sessions.test.ts` и `server/src/store.security.test.ts`. Они проверяют регистрацию и неизменяемость Curve25519-ключей устройств, допустимость резервных копий E2EE-ключей, хэширование и проверку паролей Argon2id, поведение устаревших паролей, запрет отключенных аккаунтов, изоляцию сессий, ограничение длительности remember-me сессии и миграцию учетных данных без сброса администраторских паролей.

Команда `npm test -w server` была выполнена успешно: 8 тестов пройдено, 0 тестов завершились ошибкой. Команда `npm run build` также выполнена успешно: клиент собран Vite, сервер скомпилирован TypeScript-компилятором. Автоматизированных React-тестов в проекте не обнаружено; для клиентской части применялась ручная проверка через браузер, локальный dev-сервер и production-сборку.

В ходе отладки устранялись ошибки, связанные с отображением интерфейса, работой тем, контекстным меню, удалением сообщений, показом медиа, записью голосовых сообщений и видеокружков, подтверждением почты, настройкой DNS/SSL и переносом состояния в MySQL. Наличие миграционных скриптов `migrate-video-notes.mjs`, `migrate-voice-messages.mjs` и `remove-users-`

without-email.mjs подтверждает, что проект развивался итерационно и требовал сопровождения данных.

3.10 Документирование проекта

Документация проекта представлена README.md, где описана структура проекта, запуск в режиме разработки, production-сборка и основные функции. В DESKTOP_RELEASE.md и desktop/README.md зафиксированы материалы по desktop-направлению. В deploy находятся конфигурации для Nginx, systemd, SSH hardening и резервного копирования. Файл server/.env.example описывает переменные окружения для почты, CORS, push-уведомлений и других параметров.

3.11 Список выполненных работ

1. Проведен анализ предметной области и требований к веб-мессенджеру.
2. Создана структура монорепозитария с рабочими областями client и server.
3. Разработан интерфейс входа, регистрации, сброса пароля и подтверждения кода из письма.
4. Реализован интерфейс списка чатов, окна переписки, профилей, модальных окон и просмотра изображений.
5. Настроен обмен сообщениями в реальном времени через Socket.IO.
6. Реализованы личные чаты, группы, каналы и избранное.
7. Добавлены реакции, ответы, пересылка, редактирование, удаление, закрепление сообщений и индикатор печати.
8. Добавлена отправка изображений, файлов, голосовых сообщений и видеокружков.
9. Реализованы голосовые и видеозвонки с использованием WebRTC-сигналинга.
10. Реализованы push-уведомления через Service Worker и web-push.
11. Настроено хранение данных в MySQL и служебные таблицы для сессий и E2EE.

12. Реализованы меры безопасности: Argon2id, JWT-сессии, rate limiting, Helmet, email-подтверждение, админ-права.
13. Подготовлены конфигурации Nginx, systemd и резервного копирования.
14. Проведена сборка проекта и запуск автоматизированных серверных тестов.
15. Подготовлен официальный отчет по производственной практике на основе исходного кода проекта.

4 Охрана труда

При выполнении работ по разработке веб-приложения основным рабочим инструментом являлся персональный компьютер. Работа за компьютером относится к видам деятельности, требующим соблюдения требований эргономики, электробезопасности и рационального режима труда и отдыха. Несоблюдение данных требований может привести к повышенной утомляемости, перенапряжению зрения, нарушению осанки и снижению работоспособности.

4.1 Требования к рабочему месту

Рабочее место должно обеспечивать удобное положение пользователя. Стол должен иметь достаточную площадь для монитора, клавиатуры, мыши и документов. Кресло должно регулироваться по высоте и поддерживать спину. Монитор следует размещать на расстоянии примерно 50–70 см от глаз, верхняя граница экрана должна находиться на уровне глаз или немного ниже. Клавиатура и мышь должны располагаться так, чтобы кисти рук не находились в постоянном напряжении.

4.2 Требования к освещению

Освещение должно быть равномерным, без резких бликов на экране. Рабочее место рекомендуется располагать так, чтобы естественный свет падал сбоку. При недостаточном освещении необходимо использовать искусственный источник света, не направленный прямо в глаза пользователя и не создающий отражений на мониторе.

4.3 Режим труда и отдыха

При длительной работе за компьютером необходимо делать регулярные перерывы. Рекомендуется каждые 45–60 минут выполнять короткий перерыв продолжительностью 5–10 минут, а также периодически выполнять упражнения для глаз и кистей рук. Во время практики особенно важно чередовать программирование, чтение документации, тестирование и оформление отчета, чтобы снизить монотонную нагрузку.

4.4 Электробезопасность

Перед началом работы необходимо убедиться в исправности кабелей питания, розеток, сетевых фильтров и оборудования. Запрещается работать с поврежденными проводами, разбирать оборудование под напряжением, ставить жидкости рядом с системным блоком, ноутбуком или периферией. Подключение оборудования должно выполняться только сухими руками. При появлении запаха гари, искрения или нестабильной работы устройства следует прекратить работу и отключить питание.

4.5 Правила работы с компьютерной техникой

Не допускается установка неизвестного программного обеспечения без проверки источника. При работе с серверными учетными данными, ключами почтовых сервисов, `JWT_SECRET`, `DATABASE_URL` и другими секретами необходимо исключать их публикацию в открытом репозитории. Резервные копии следует хранить в защищенной директории с ограниченными правами доступа. В проекте эти требования частично отражены через `server/.env.example`, `systemd UMask=0077` и `backup`-скрипт, создающий файлы с правами `0600/0700`.

4.6 Профилактика утомления

Для профилактики утомления рекомендуется соблюдать правильную посадку, регулярно проветривать помещение, использовать комфортную яркость экрана, избегать длительной работы в полной темноте, выполнять гимнастику для

глаз и разминку мышц шеи, плеч и кистей. При разработке интерфейса также следует учитывать зрительную нагрузку конечных пользователей: достаточный контраст текста, читаемый размер шрифта, понятные состояния кнопок и отсутствие резких мигающих эффектов.

Вывод

В ходе производственной практики была выполнена разработка полнофункционального веб-мессенджера «БренксЧат». Проект включает клиентскую часть на React и TypeScript, серверную часть на Node.js, Express и Socket.IO, постоянное хранение данных в MySQL, авторизацию с подтверждением почты, работу с сообщениями в реальном времени, медиафайлами, голосовыми сообщениями, видеокружками, WebRTC-звонками, push-уведомлениями и административными функциями.

В процессе практики были закреплены навыки проектирования структуры веб-приложения, разработки REST API, использования WebSocket, работы с MySQL, настройки сессий и ролей, реализации безопасного хранения паролей, интеграции почтовых уведомлений, настройки production-сборки и серверного развертывания. Особое значение имело исправление ошибок и итерационная доработка проекта, так как мессенджер содержит множество взаимосвязанных частей: интерфейс, realtime-обмен, хранение данных, авторизацию, медиа и звонки.

Разработка мессенджера позволила получить практический опыт, максимально приближенный к реальной деятельности веб-разработчика. Были сформированы компетенции по анализу требований, проектированию архитектуры, написанию клиентского и серверного кода, работе с базами данных, тестированию, отладке, документированию и обеспечению безопасности веб-приложений. Поставленные задачи практики выполнены, а полученный результат может быть использован как основа для дальнейшего развития проекта: масштабирования БД, улучшения UI/UX, расширения E2EE на медиа, повышения надежности звонков через TURN и выпуска desktop-клиента.

Список использованных источников

1. Исходные материалы проекта «БренксЧат»: README.md, package.json, client/src, server/src, deploy, desktop. — Локальный репозиторий проекта, 2026.
2. React Documentation [Электронный ресурс]. — Режим доступа: <https://react.dev/> (дата обращения: 16.06.2026).
3. TypeScript Documentation [Электронный ресурс]. — Режим доступа: <https://www.typescriptlang.org/docs/> (дата обращения: 16.06.2026).
4. Vite Guide [Электронный ресурс]. — Режим доступа: <https://vite.dev/guide/> (дата обращения: 16.06.2026).
5. Tailwind CSS Documentation [Электронный ресурс]. — Режим доступа: <https://tailwindcss.com/docs> (дата обращения: 16.06.2026).
6. Express Documentation [Электронный ресурс]. — Режим доступа: <https://expressjs.com/> (дата обращения: 16.06.2026).
7. Socket.IO Documentation [Электронный ресурс]. — Режим доступа: <https://socket.io/docs/> (дата обращения: 16.06.2026).
8. Node.js Documentation [Электронный ресурс]. — Режим доступа: <https://nodejs.org/docs/> (дата обращения: 16.06.2026).
9. MySQL Reference Manual [Электронный ресурс]. — Режим доступа: <https://dev.mysql.com/doc/> (дата обращения: 16.06.2026).
10. Nginx Documentation [Электронный ресурс]. — Режим доступа: <https://nginx.org/en/docs/> (дата обращения: 16.06.2026).
11. MDN Web Docs: WebRTC API [Электронный ресурс]. — Режим доступа: https://developer.mozilla.org/docs/Web/API/WebRTC_API (дата обращения: 16.06.2026).
12. MDN Web Docs: Service Worker API [Электронный ресурс]. — Режим доступа: https://developer.mozilla.org/docs/Web/API/Service_Worker_API (дата обращения: 16.06.2026).
13. OWASP Cheat Sheet Series: Password Storage Cheat Sheet [Электронный ресурс]. — Режим доступа:

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
(дата обращения: 16.06.2026).

14. libsodium Documentation [Электронный ресурс]. — Режим доступа:
<https://doc.libsodium.org/> (дата обращения: 16.06.2026).

15. RFC 7519. JSON Web Token (JWT) [Электронный ресурс]. — Режим
доступа: <https://www.rfc-editor.org/rfc/rfc7519> (дата обращения: 16.06.2026).